Министерство образования Республики Беларусь

Учреждение образования

«Брестский государственный технический университет»

Кафедра интеллектуальных информационных технологий

Лабораторная работа №5

По дисциплине «Интеллектуальный анализ данных»

Тема: «Деревья решений»

Выполнил:

Студент 4 курса

Факультета ЭИС

Группы ИИ-26

Шоева Е.Т.

Проверил:

Никитин С.К.

Брест 2026

Цель работы: На практике сравнить работу нескольких алгоритмов одиночного дерева решений, случайного леса и бустинга для деревьев решений.

Ход работы

Задачи:

1. Загрузить датасет по варианту;
2. Разделить данные на обучающую и тестовую выборки;
3. Обучить на обучающей выборке одиночное дерево, случайный лес и реализовать бустинг для решающих деревьев (AdaBoost, CatBoost, XGBoost);
4. Оценить точность каждой модели на тестовой выборке;
5. Сравнить результаты, сделать выводы о применимости каждого метода для данного набора данных.

Вариант 11

- `Bank Marketing`
- Предсказать, подпишется ли клиент на срочный вклад
- **Задания:**
    1. Загрузите данные и преобразуйте категориальные признаки;
    2. Разделите выборку;
    3. Обучить на обучающей выборке одиночное дерево, случайный лес и реализовать бустинг для решающих деревьев (AdaBoost, CatBoost, XGBoost);
    4. Сравните модели по `F1-score` из-за дисбаланса классов;
    5. Определите, какая модель предлагает лучший компромисс для выявления потенциальных клиентов.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier
from sklearn.metrics import f1_score, precision_score, recall_score,
confusion_matrix
from xgboost import XGBClassifier
from catboost import CatBoostClassifier


df = pd.read_csv('bank.csv', sep=None, engine='python')
df.columns = [c.strip().strip('"') for c in df.columns]


target_col = 'y' if 'y' in df.columns else 'deposit'
y =
df[target_col].astype(str).str.strip().str.strip('"').str.lower().map({'yes': 1,
'no': 0}).values
X = df.drop(columns=[target_col])
```

```
    print("Размер данных:", X.shape)
    print("Доля класса 'yes':", y.mean())

    cat_cols = X.select_dtypes(include=['object', 'str']).columns
    X = pd.get_dummies(X, columns=cat_cols, drop_first=True).astype(float)

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42, stratify=y
    )

    neg, pos = np.sum(y_train == 0), np.sum(y_train == 1)

    models = {
        'Решающее дерево': DecisionTreeClassifier(max_depth=6,
class_weight='balanced', random_state=42),
        'Случайный лес': RandomForestClassifier(n_estimators=300,
class_weight='balanced', random_state=42, n_jobs=-1),
        'AdaBoost': AdaBoostClassifier(n_estimators=300, learning_rate=0.5,
random_state=42),
        'XGBoost': XGBClassifier(n_estimators=300, max_depth=5, learning_rate=0.1,
                                 scale_pos_weight=neg / pos, eval_metric='logloss',
random_state=42),
        'CatBoost': CatBoostClassifier(iterations=300, depth=6, learning_rate=0.1,
                                       auto_class_weights='Balanced', verbose=0,
random_seed=42),
    }

    results = {}
    cms = {}

    for name, model in models.items():
        model.fit(X_train, y_train)
        preds = model.predict(X_test)
        results[name] = {
            'F1': f1_score(y_test, preds),
            'Precision': precision_score(y_test, preds),
            'Recall': recall_score(y_test, preds),
        }
        cms[name] = confusion_matrix(y_test, preds)
        print(f"\n{name}")
```

```
        print(f"F1 = {results[name]['F1']:.4f}, Precision =
{results[name]['Precision']:.4f}, Recall = {results[name]['Recall']:.4f}")
        print("Confusion matrix:\n", cms[name])

    res_df = pd.DataFrame(results).T.sort_values('F1', ascending=False)
    print("\nИтоговая таблица (по убыванию F1):")
    print(res_df.round(4))

    best_f1 = res_df['F1'].idxmax()
    best_recall = res_df['Recall'].idxmax()
    print(f"\nЛучший F1: {best_f1} ({res_df.loc[best_f1, 'F1']:.4f})")
    print(f"Лучший Recall: {best_recall} ({res_df.loc[best_recall, 'Recall']:.4f})")

    plt.figure(figsize=(10, 6))
    res_df.plot(kind='bar', ax=plt.gca())
    plt.ylabel('Значение метрики')
    plt.title('Сравнение моделей (класс "yes")')
    plt.xticks(rotation=20)
    plt.tight_layout()
    plt.savefig('lab5_metrics.png')
    plt.show()

    fig, axes = plt.subplots(1, len(cms), figsize=(4.5 * len(cms), 4))
    for ax, (name, cm) in zip(axes, cms.items()):
        sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=ax, cbar=False)
        ax.set_title(name)
        ax.set_xlabel('Предсказано')
        ax.set_ylabel('Истина')
    plt.tight_layout()
    plt.savefig('lab5_confusion_matrices.png')
    plt.show()
```

```
Размер данных: (11162, 16)
Доля класса 'yes': 0.4738398136534671

Решающее дерево
F1 = 0.8021, Precision = 0.8036, Recall = 0.8006
Confusion matrix:
 [[968 207]
 [211 847]]

Случайный лес
F1 = 0.8597, Precision = 0.8247, Recall = 0.8979
Confusion matrix:
 [[973 202]
 [108 950]]

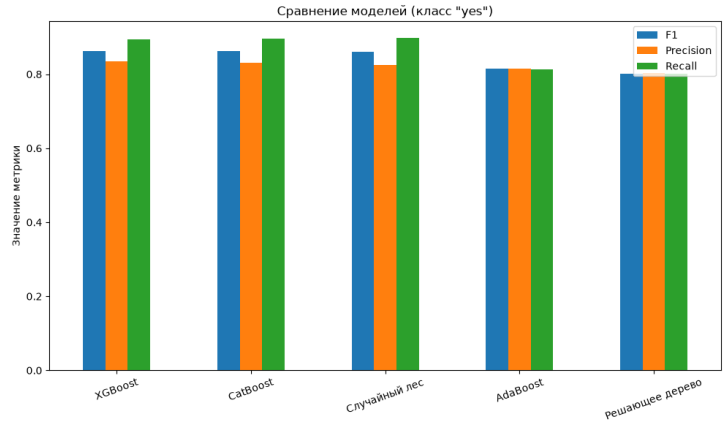
AdaBoost
F1 = 0.8142, Precision = 0.8146, Recall = 0.8138
Confusion matrix:
 [[979 196]
 [197 861]]

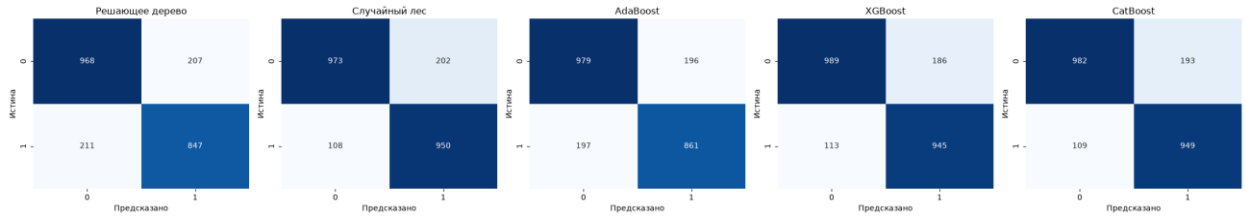
XGBoost
F1 = 0.8634, Precision = 0.8355, Recall = 0.8932
Confusion matrix:
 [[989 186]
 [113 945]]
```

```
CatBoost
F1 = 0.8627, Precision = 0.8310, Recall = 0.8970
Confusion matrix:
 [[982 193]
 [109 949]]

Итоговая таблица (по убыванию F1):
                      F1  Precision  Recall
XGBoost           0.8634     0.8355  0.8932
CatBoost          0.8627     0.8310  0.8970
Случайный лес     0.8597     0.8247  0.8979
AdaBoost          0.8142     0.8146  0.8138
Решающее дерево   0.8021     0.8036  0.8006

Лучший F1: XGBoost (0.8634)
Лучший Recall: Случайный лес (0.8979)
```





К датасету Bank Marketing (11162 объекта, 16 признаков, целевая переменная: подписка клиента на срочный вклад) применены пять моделей: одиночное решающее дерево, случайный лес, AdaBoost, XGBoost и CatBoost. Категориальные признаки были закодированы методом one-hot, выборка разделена на обучающую и тестовую в соотношении 80/20 со стратификацией. Доля класса «yes» составила около 47%, то есть классы близки к сбалансированным, но сравнение проводилось по F1-score, как требует задание.

➢ Одиночное дерево показало худший результат (F1 = 0.8021, Recall = 0.8006): модель простая и интерпретируемая, но пропускает 211 потенциальных клиентов.

➢ AdaBoost улучшил результат лишь незначительно (F1 = 0.8142) и заметно уступил остальным ансамблям.

➢ Случайный лес (F1 = 0.8597, Recall = 0.8979), XGBoost (F1 = 0.8634, Recall = 0.8932) и CatBoost (F1 = 0.8627, Recall = 0.8970) показали близкие и наилучшие результаты. Различие между ними по F1 составляет менее 0.004, что соответствует нескольким объектам тестовой выборки и находится в пределах случайного разброса.

➢ Лучший компромисс между точностью и полнотой даёт XGBoost: наибольший F1 и наименьшее число ложных срабатываний среди лидеров (186 против 193 у CatBoost и 202 у леса) при высоком Recall. Если приоритетом является максимальное выявление потенциальных клиентов, можно выбрать CatBoost или случайный лес, у которых меньше всего пропусков (109 и 108), ценой чуть большего числа лишних контактов.

Итог: ансамблевые методы (случайный лес, XGBoost, CatBoost) заметно эффективнее одиночного дерева и AdaBoost; для выявления потенциальных клиентов рекомендуется XGBoost, а CatBoost является практически равноценной альтернативой.

Вывод: На практике сравнила работу нескольких алгоритмов одиночного дерева решений, случайного леса и бустинга для деревьев решений.